

I N N O V A N E X U S

Documentação Técnica

Conectando ideias a resultados

Sprint 1 · Plataforma de Gestão de Inovação Corporativa

Stack: Android · Kotlin · Clean Architecture · MVVM · Firebase

Data: Maio de 2026

Versão do documento: 2.0 — Front-end + Back-end unificados

Sumário

PARTE I — VISÃO GERAL

Introdução

Objetivo do Projeto

Perfis de Usuário

Tecnologias Utilizadas

Telas Implementadas

Diferenciais

PARTE II — ARQUITETURA

Visão de Arquitetura

Padrões Adotados

Estrutura da Aplicação

PARTE III — FUNCIONALIDADES DE NEGÓCIO

Priorização de Ideias

Gamificação e Reconhecimento

Fluxo de Aprovação

Dashboard Executivo

PARTE IV — BACK-END E INTEGRAÇÃO

Back-end Mock-first

Integração com Firebase

Firestore Security Rules

Cloud Functions de Automação

Operando sem Cloud Functions (Plano Spark)

PARTE V — APÊNDICES

Como Rodar o Projeto

ADR-001: Adoção do Firebase como BaaS

Histórico da Sprint 1

Conclusão

Introdução

Slogan: *Conectando ideias a resultados.*

O **InnovaNexus** é uma plataforma mobile Android desenvolvida para gestão da inovação corporativa. O sistema conecta operadores, gestores e lideranças em um fluxo contínuo de criação, avaliação e acompanhamento de ideias e projetos, transformando contribuições individuais em resultados mensuráveis para o negócio.

A documentação a seguir consolida, em um único material, tanto a visão de produto e front-end quanto a arquitetura de back-end, integrações de nuvem e decisões técnicas que sustentam a Sprint 1.

Objetivo do Projeto

Permitir o gerenciamento de estratégias corporativas, ideias de inovação, projetos e indicadores de desempenho em uma única aplicação nativa Android, com três perfis de uso distintos e fluxos de aprovação automatizados entre eles.

A plataforma atende três grandes necessidades:

- **Capturar ideias da operação** de forma simples e rápida, com pontuação e reconhecimento.
- **Priorizar e aprovar ideias** com critérios objetivos (ICE + votos), evitando decisões puramente subjetivas.
- **Acompanhar o impacto** das ideias aprovadas através de um dashboard executivo com ROI, economia gerada e produtividade.

Perfis de Usuário

A aplicação foi modelada em torno de três perfis que refletem o fluxo real de inovação dentro de uma empresa:

Perfil	Responsabilidade	Telas principais
Operador	Cadastra ideias, apoia ideias de colegas, acompanha seu próprio histórico e posição no ranking.	Home Operador, Nova Ideia, Minhas Ideias, Ranking
Gestor	Avalia, prioriza, aprova ou rejeita ideias. Gerencia projetos derivados das ideias aprovadas.	Home Gestor, Gestão de Ideias, Projetos, Detalhe da Ideia
Liderança	Define estratégias corporativas e acompanha indicadores agregados de desempenho.	Dashboard Executivo, Gestão Estratégica

A separação por perfil é aplicada tanto na navegação quanto nas regras de negócio: o que cada usuário pode ler ou escrever é controlado em camada de back-end (Firestore Security Rules) e refletido na UI por meio de Fragments dedicados.

Tecnologias Utilizadas

A stack foi escolhida para equilibrar produtividade, padrão de mercado Android e baixa fricção de manutenção.

Camada	Tecnologias
Linguagem e IDE	Kotlin, Android Studio
UI	XML Layouts, Material Design 3, ViewBinding, Navigation Component
Arquitetura	MVVM (Model-View-ViewModel) na apresentação, Clean Architecture nas camadas inferiores
Concorrência	Kotlin Coroutines, <code>StateFlow</code>
Rede	Retrofit, Gson, OkHttp
Mock de back-end	MockWebServer (debug)
Persistência local	Room Database
Listas	RecyclerView
Gráficos	MPAndroidChart
Back-end de nuvem	Firebase (Authentication, Cloud Firestore, Cloud Messaging, Cloud Functions)
Testes	JUnit 4, MockK

A combinação de Retrofit + Room + Firebase permite que o app opere em três modos: 100% local (Room), mock-first (Retrofit contra um servidor HTTP local) e produção (Firebase). A escolha entre os modos é feita em tempo de build.

Telas Implementadas

A navegação é estruturada com **Navigation Component** e segue um único grafo, com pontos de entrada distintos para cada perfil após o login.

Grupo	Telas
Comum	Splash, Login, Perfil

Operador	Home Operador, Estratégias, Nova Ideia, Minhas Ideias, Ranking
Gestor	Home Gestor, Gestão de Ideias, Detalhe da Ideia, Projetos, Editar Projeto
Liderança	Dashboard Executivo, Gestão Estratégica

A interface adota **Material Design 3** com foco em clareza visual, acessibilidade e consistência entre perfis. Estados de loading, erro e vazio são tratados explicitamente em cada ViewModel via `StateFlow`, evitando que a UI fique em estados inconsistentes durante operações assíncronas.

PARTE I — VISÃO GERAL

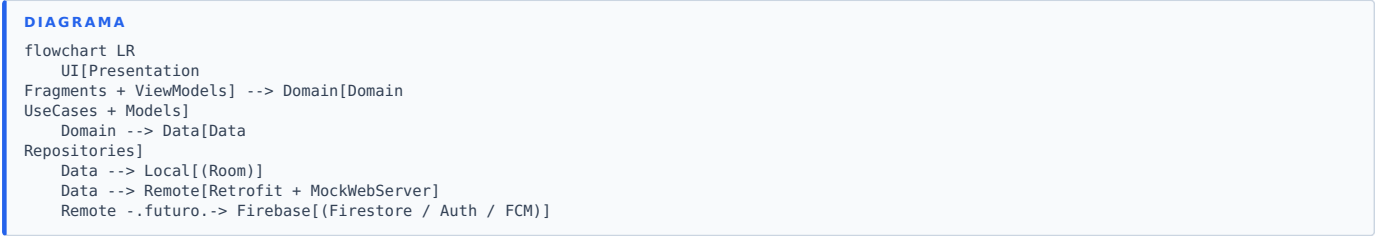
Diferenciais

O projeto se diferencia por entregar, em uma única Sprint, uma experiência ponta a ponta com:

- **Dashboard executivo** com ROI, economia gerada, produtividade e gráfico de ideias por mês.
- **Gamificação por ranking** com pódio dinâmico (ouro/prata/bronze) e pontuação por evento.
- **Múltiplos perfis de acesso** com regras de negócio próprias em cada camada.
- **Arquitetura escalável** baseada em Clean Architecture, com camadas independentes e testáveis.
- **Integração com Firebase** para autenticação real, persistência em nuvem e notificações.
- **Modo offline-first**: o app funciona mesmo sem conectividade, sincronizando quando possível.

Visão de Arquitetura

A aplicação adota **Clean Architecture** com três camadas claramente separadas. A camada de apresentação usa o padrão **MVVM**, enquanto domínio e dados seguem a separação clássica de regras de negócio versus orquestração de fontes.



A camada de **apresentação** organiza-se por papel de usuário, com pacotes `operator/`, `manager/` e `leadership/`. Cada ViewModel expõe um `StateFlow` com o estado da tela e recebe eventos do Fragment correspondente.

A camada de **domínio** é completamente desacoplada do Android: contém apenas modelos de dados puros (`Idea`, `Project`, `User`, `Strategy`) e UseCases que implementam as regras de negócio. Isso torna a lógica testável em JVM, sem necessidade de instrumentação.

A camada de **dados** centraliza Room (persistência local), Retrofit (comunicação HTTP) e as fontes Firebase. Os repositórios escondem essa complexidade dos UseCases e implementam a estratégia *offline-first*: tenta o remoto, cai no local em caso de falha, mantém Room como fonte única de verdade para a UI.

Padrões Adotados

Quatro padrões orientam decisões de implementação em todo o projeto:

Padrão	Aplicação
Offline-first	Repositórios tentam o remoto e, em caso de falha, recorrem ao Room. A UI continua funcional mesmo sem rede.
Single source of truth	Room é a fonte que a UI observa via <code>Flow</code> . Operações de rede sincronizam o Room; a UI nunca lê direto de Retrofit/Firebase.
Sealed interfaces	Estados e eventos dos ViewModels são modelados como <code>sealed interface State</code> / <code>sealed interface Event</code> , eliminando estados inválidos em tempo de compilação.
Injeção de dependência manual	Feita no <code>InnovaNexusApplication</code> . Suficiente para a Sprint 1; pode evoluir para Hilt ou Koin se o projeto crescer.

Esses padrões habilitam dois efeitos práticos importantes: a substituição de Retrofit por Firestore não exigiu mudanças em UseCases nem em Fragments, e os testes unitários da camada de domínio rodam sem precisar de emulador Android.

Estrutura da Aplicação

A organização dos pacotes reflete diretamente a arquitetura:

- `presentation/` — Fragments por papel (operator, manager, leadership), ViewModels com `StateFlow`, Adapters de `RecyclerView` e bindings de UI.
- `domain/model/` — Entidades puras de domínio (sem dependência de Android).
- `domain/usecase/` — Regras de negócio implementadas como UseCases atômicos (ex.: `CreateIdeaUseCase`, `ApproveIdeaUseCase`, `ScoreIdeaUseCase`).
- `data/local/` — Room: entidades, DAOs e o `AppDatabase`.
- `data/remote/` — Retrofit (`ApiService` + DTOs), `MockApiServer`, interfaces `RemoteDataSource` e implementações Firebase.
- `data/mapper/` — Conversões DTO ↔ Entity ↔ Domain, evitando vazamento de tipos de uma camada para outra.
- `data/repository/` — Implementação dos repositórios, com estratégia offline-first.

A interface `RemoteDataSource` é o ponto de extensão central: trocar Retrofit por Firestore foi feito apenas substituindo implementações dessa interface, sem qualquer alteração em UseCases ou camada de UI.

Priorização de Ideias

A Sprint 1 exige dinâmicas diferenciadas para o filtro e priorização das ideias. O InnovaNexus adota o método **ICE + voteBoost**, combinando avaliação técnica do gestor com o engajamento dos pares.

Fórmula aplicada:

ICE = impacto × confiança × facilidade (cada fator em escala 1 a 5) **voteBoost** = quantidade de votos × peso do voto (peso padrão = 5)
score = ICE + voteBoost, limitado entre 0 e 1000.

Origem dos fatores:

Fator	De onde vem
Impacto	Derivado do rótulo de impacto da ideia (Alto = 5, Médio = 3, Baixo = 1) ou definido manualmente pelo gestor.
Confiança	Inicia em 3 e pode ser ajustado pelo gestor durante a análise.
Facilidade	Inicia em 3 e pode ser ajustado pelo gestor.
Votos	Cada operador pode apoiar uma ideia. Voto duplicado é bloqueado pelo <code>VoteIdeaUseCase</code> .

Exemplos de cálculo:

Caso	I	C	E	Votos	Score
Ideia "Alto impacto" recém-criada	5	3	3	0	45
Mesma ideia com 4 apoios	5	3	3	4	65
Ideia "Baixo impacto" sem apoios	1	3	5	0	15
Ideia "Alto impacto", alta confiança, 10 apoios	5	5	3	10	125

Auto-promoção: quando uma ideia em análise acumula 5 ou mais votos, o sistema marca-a como candidata à promoção automática. A regra está disponível para ser conectada a uma Cloud Function quando o ambiente Firebase estiver ativo.

Consulta priorizada: as listas exibidas ao gestor são ordenadas por score (ICE descendente, depois votos), garantindo que ideias com maior potencial apareçam primeiro.

Gamificação e Reconhecimento

A Sprint pede reconhecimento dentro do sistema para quem mais contribui ou tem ideias escolhidas. O InnovaNexus implementa uma economia de pontos vinculada a eventos do fluxo de ideias.

Regras de pontuação:

Evento	Pontos	Onde é disparado
Cadastrar uma ideia	+10	<code>CreateIdeaUseCase</code>
Ter ideia aprovada	+50	<code>ApproveIdeaUseCase</code>
Ideia convertida em projeto	+100 (cumulativo)	<code>ApproveIdeaUseCase</code>
Receber um voto de apoio	+2	<code>VoteIdeaUseCase</code> (pontua o autor da ideia, não o votante)

A pontuação é acumulada no campo `pontuacao` de cada usuário e exposta através do `UserRepository.observeRanking()`, que retorna a lista ordenada por pontos.

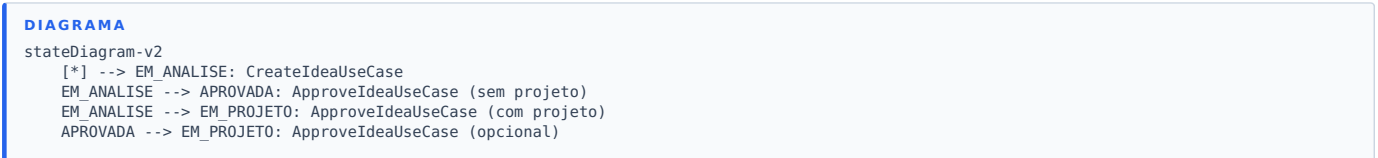
Ranking visual:

- O **top 1** recebe medalha de ouro.
- O **top 2**, medalha de prata.
- O **top 3**, medalha de bronze.
- Posições a partir de 4 aparecem na lista regular.
- Se houver menos de três usuários, os slots vazios do pódio são automaticamente escondidos.

Por que pontuar o autor pelo voto recebido: essa escolha reforça o efeito de rede — quem propõe ideias relevantes para os pares acumula vantagem natural. O bloqueio de voto duplicado evita fraude trivial. Quando as Cloud Functions estiverem ativas, uma função `onIdeaStatusChange` poderá recalcular a pontuação no servidor, eliminando qualquer chance de inconsistência entre dispositivos.

Fluxo de Aprovação

O ciclo de vida de uma ideia atravessa quatro estados possíveis, todos modelados explicitamente no domínio:



```
EM_ANALISE --> REJEITADA: RejectIdeaUseCase
EM_PROJETO --> [*]
REJEITADA --> [*]
```

Automação embutida na aprovação. Quando o gestor aprova uma ideia indicando que ela deve virar projeto, o `ApproveIdeaUseCase` executa cinco passos em uma única invocação atômica:

1. Atualiza o status da ideia para *aprovada*.
2. Credita +50 pontos ao autor e incrementa seu contador de ideias aprovadas.
3. Cria um novo projeto em status *planejamento*, com referência à ideia de origem.
4. Atualiza o status da ideia para *em projeto*.
5. Credita +100 pontos adicionais ao autor pela conversão.

Os cinco passos rodam dentro do mesmo escopo de coroutine. Qualquer falha intermediária propaga uma exceção e impede passos subsequentes, evitando estados inconsistentes (uma ideia "em projeto" sem projeto correspondente, por exemplo).

Reflexo na UI. Na tela de detalhe da ideia, o gestor vê o cartão da ideia com score, número de votos e três botões: Apoiar, Aprovar, Rejeitar. Os botões de aprovar e rejeitar são automaticamente ocultados quando a ideia já está em estado final.

Evolução para Cloud Functions. Quando o ambiente Firebase estiver com Cloud Functions ativas, essa mesma lógica será espelhada no servidor através de um trigger no evento `onUpdate` da coleção de ideias, garantindo que a regra valha mesmo se o cliente Android estiver desatualizado.

PARTE III — FUNCIONALIDADES DE NEGÓCIO

Dashboard Executivo

O Dashboard Executivo é acessível ao perfil de liderança e concentra os indicadores agregados de desempenho da plataforma.

Métricas exibidas:

Métrica	Como é calculada
ROI total	$(\Sigma \text{ retorno} - \Sigma \text{ investimento}) / \Sigma \text{ investimento} \times 100$
Economia gerada	$\Sigma \text{ retorno} - \Sigma \text{ investimento}$
Ideias aprovadas	Total de ideias com status <i>aprovada</i>
Projetos ativos	Total de projetos em <i>execução</i>
Produtividade percentual	$(\text{ideias aprovadas} / \text{total de ideias}) \times 100$
Ideias por mês	Agrupamento das ideias aprovadas ou em projeto pelo mês de envio

Estratégia offline-first do dashboard. O `DashboardRepository` tenta primeiro buscar os dados agregados no remoto. Se a chamada falhar, ou se a resposta não trouxer o histórico mensal, o repositório recorre a um cálculo local que agrega diretamente os dados do Room. Esse fallback garante que o gráfico de ideias por mês nunca apareça vazio sem motivo.

Visualizações:

Tipo de gráfico	Conteúdo
Gráfico de barras	ROI por projeto
Gráfico de pizza	Projetos por status (Planejamento, Execução, Concluído, Pausado)
Gráfico de linha	Ideias aprovadas por mês (12 pontos: janeiro a dezembro)

Defesa contra dados ruins: a função que monta o gráfico de linha verifica o tamanho da lista recebida. Se o array tiver tamanho diferente de 12, é substituído por uma lista de zeros — o gráfico nunca lança exceção, mesmo com dados incompletos.

Back-end Mock-first

Durante a Sprint 1 não foi provisionado um servidor real. Em vez disso, o app consome uma **API REST simulada** através do `MockWebServer`, embutido no próprio binário de debug. Toda a interface Retrofit faz chamadas HTTP reais — apenas o servidor é local.

Por que mock-first. Essa decisão permitiu que o time iterasse no front-end e nas regras de negócio sem depender da infraestrutura de back-end, ao mesmo tempo em que mantém o contrato HTTP fiel ao que será usado em produção. Quando o ambiente Firebase é ativado, basta trocar a implementação concreta da interface `RemoteDataSource` — repositórios, UseCases e Fragments continuam idênticos.

Endpoints simulados:

Método	Recurso	Resposta
POST	Login	Token simulado e papel inferido do e-mail
GET	Estratégias, Ideias, Projetos	Lista de fixtures
POST	Estratégias, Ideias, Projetos	Echo do corpo com id gerado
PUT	Estratégias, Ideias, Projetos	Echo
DELETE	Estratégias	204 (No Content)
GET	Dashboard	Estrutura completa com <code>ideiasPerMonth</code>

Sequência típica de uma chamada:

DIAGRAMA

```
sequenceDiagram
    participant Fragment->>ViewModel: submit()
    participant ViewModel->>UseCase: invoke(input)
    participant UseCase->>Repository: createIdea(idea)
    participant Repository->>RemoteDataSource: create(dto)
    participant RemoteDataSource->>ApiService: POST /ideas
    participant ApiService->>MockApiServer: HTTP request
    participant MockApiServer-->>ApiService: 200 + JSON
    participant ApiService-->>Repository: DTO
    participant Repository->>Room: insert(entity)
    participant Repository-->>UseCase: Idea
    participant UseCase-->>ViewModel: Success
    participant ViewModel-->>Fragment: State.Success
```

Integração com Firebase

Firebase foi escolhido entre os quatro BaaS sugeridos pela Sprint (Firebase, OneSignal, Backendless, AWS Amplify) por ser o único que cobre, em um único SDK Kotlin-first, todos os requisitos: autenticação real, persistência em nuvem, notificações push e automações server-side.

Comparação dos BaaS avaliados:

Critério	Firebase	OneSignal	Backendless	AWS Amplify
Autenticação	✓	✗	✓	✓
Persistência	✓ Firestore	✗	✓	✓
Push notifications	✓ FCM	✓	✓	✓
Cloud Functions	✓	✗	⚠ Codeless	✓ Lambda
SDK Kotlin Android	✓ KTX	✓ só push	⚠	⚠
Free tier para MVP	✓ Spark	✓	⚠	⚠

Componentes Firebase utilizados:

Componente	Função no projeto
Firebase Authentication	Login real com e-mail/senha; o token JWT acompanha cada chamada ao Firestore.
Cloud Firestore	Coleções <code>users</code> , <code>ideas</code> , <code>projects</code> , <code>strategies</code> , <code>dashboard</code> .
Firebase Cloud Messaging	Notificações push para o autor quando a ideia é aprovada, rejeitada ou recebe votos relevantes.
Firebase Analytics	Eventos de uso automáticos via SDK.

Switch entre back-ends. O app alterna entre os modos *mock* e *Firebase* em tempo de compilação, controlado por duas flags em `BuildConfig`. Sem o arquivo `google-services.json` em `app/`, o projeto compila sem qualquer dependência Firebase. Com o arquivo presente e a flag de mock desligada, o app passa a apontar para o ambiente real automaticamente.

Sequência de autenticação e leitura:

DIAGRAMA

```
sequenceDiagram
    participant App as Android
    participant Auth as FirebaseAuth
```



```
participant FS as Cloud Firestore

App->>Auth: signInWithEmailAndPassword(email, senha)
Auth-->>App: FirebaseUser (uid + JWT)
App->>FS: collection("ideas").get()
Note over App,FS: SDK anexa JWT automaticamente
FS->>FS: Security Rules avaliam request.auth
FS->>App: QuerySnapshot
```

PARTE IV — BACK-END E INTEGRAÇÃO

Firestore Security Rules

As **Security Rules** do Firestore substituem um back-end intermediário: o cliente Android chama o Firestore diretamente, e o servidor decide se permite a operação com base em quem fez a chamada (`request.auth`) e no estado do documento (`resource.data`).

Mapa de permissões adotado para a Sprint 1:

Coleção	Leitura	Escrita
users	Qualquer usuário autenticado	Apenas o próprio usuário (uid igual ao id do documento)
ideas	Qualquer usuário autenticado	Criar: qualquer autenticado. Atualizar: autor ou perfis gestor/liderança.
projects	Qualquer usuário autenticado	Apenas perfis gestor ou liderança
strategies	Qualquer usuário autenticado	Apenas perfil liderança
dashboard	Qualquer usuário autenticado	Nunca via cliente (apenas Cloud Function)

Como o papel do usuário é checado. A regra faz uma leitura adicional ao documento do usuário em `users/{uid}` para descobrir o papel armazenado em `role`. Esse padrão é simples e suficiente para a Sprint, mas tem um custo: cada operação que dependa do papel gera uma leitura extra. Em ambientes de produção com volume, recomenda-se mover o papel para **Custom Claims** do JWT, eliminando a leitura adicional. Essa evolução está fora do escopo da Sprint 1.

Testes locais. O conjunto de regras é testável com o **Firebase Local Emulator Suite**, que sobe Firestore e Auth localmente. Os testes podem validar que um usuário com papel "gestor" não consegue criar uma estratégia (ação reservada à liderança), por exemplo, sem nunca tocar o ambiente de produção.

PARTE IV — BACK-END E INTEGRAÇÃO

Cloud Functions de Automação

As funções server-side concentram as **automações de fluxo** exigidas pela Sprint: o servidor reage a eventos do Firestore e atualiza estado ou dispara notificações sem que o cliente precise orquestrar nada.

Funções planejadas e seus gatilhos:

Função	Gatilho	Ação
onIdeaStatusChange	Atualização em <code>ideas/{id}</code>	Quando o status muda para <i>aprovada</i> : credita +50 pontos ao autor e envia notificação push. Quando muda para <i>em projeto</i> : cria documento em <code>projects/</code> , credita +100 pontos e notifica. Quando muda para <i>rejeitada</i> : envia notificação de aviso.
onIdeaVoteUpdated	Atualização em <code>ideas/{id}</code>	Quando os votos cruzam o threshold (5), notifica o autor de que a ideia ganhou destaque.
onNewUser	Criação em <code>users/{id}</code>	Registra log de auditoria.
aggregateDashboard	Cron a cada 60 minutos	Recalcula o documento <code>dashboard/current</code> agregando ideias e projetos.

Fonte autoritativa de pontuação. Hoje, sem as Cloud Functions implantadas, o cliente Android é quem credita pontos. Quando as funções forem implantadas, o cliente para de pontuar e o servidor passa a ser a fonte única — eliminando qualquer risco de duplicação ou divergência entre dispositivos.

Limitação atual. Cloud Functions exigem o plano **Blaze** do Firebase, que é pago (com cartão cadastrado), embora ofereça um free tier generoso. Para a Sprint 1, o projeto opera no plano gratuito **Spark** e usa uma estratégia alternativa, descrita a seguir, que entrega o mesmo valor sem custo.

PARTE IV — BACK-END E INTEGRAÇÃO

Operando sem Cloud Functions (Plano Spark)

Para entregar **todos os requisitos da Sprint 1** mantendo o projeto no plano Spark gratuito, foi adotada uma estratégia baseada em **snapshot listeners do Firestore** e **notificações locais** do Android.

Substituições feitas:

Função planejada (Blaze)	Substituto adotado (Spark)
Pontuação no servidor após aprovação	Cliente já aplica a pontuação dentro do <code>ApproveIdeaUseCase</code> .
Criação automática de projeto	Cliente cria o projeto na mesma transação da aprovação.
Push para o autor quando ideia muda de status	Snapshot listener no Firestore + notificação local Android.
Push ao cruzar threshold de votos	Mesmo padrão: listener + notificação local.

Cron de agregação do dashboard	<code>DashboardRepository</code> agrega os dados a cada abertura da tela.
--------------------------------	---

O mecanismo do snapshot listener. O SDK Android do Firestore permite que o app se inscreva em uma query e receba notificações em tempo real sempre que algo muda, sem custar nada além de leituras normais. Quando qualquer gestor, em qualquer dispositivo, aprova uma ideia, o Firestore propaga a mudança para o dispositivo do autor em milissegundos. O app local detecta a mudança e dispara uma notificação no sistema operacional Android. **Não há servidor envolvido.**

DIAGRAMA

```
sequenceDiagram
    participant Gestor as Gestor (device A)
    participant FS as Firestore
    participant Autor as Autor (device B, logado)

    Gestor->>FS: update ideas/{id}.status = "aprovada"
    FS->>Autor: snapshot listener dispara
    Autor->>Autor: LocalNotifier.show("Ideia aprovada!")
```

Limites desta estratégia. A solução funciona perfeitamente enquanto o autor está com o app aberto ou em segundo plano. Se o app estiver totalmente fechado por longos períodos, notificações antigas podem ser perdidas — o listener vai disparar pelo histórico recente quando o usuário reabrir. Para demonstração acadêmica, esse limite é totalmente aceitável.

Quando migrar para Blaze. A migração faria sentido se for necessário enviar push para usuários com app fechado por dias, atender mais de mil usuários ativos, executar lógica sem cliente online (lembretes diários, por exemplo) ou integrar com sistemas externos como e-mail ou Slack. Nada disso é requisito da Sprint 1.

Como Rodar o Projeto

Pré-requisitos:

- Android Studio Koala ou superior
- JDK 17 (já configurado nas opções de compilação)
- Android SDK 35 instalado
- Emulador Android API 24 ou superior (também aceita device físico)

Clonar e abrir. Após clonar o repositório, abrir a pasta no Android Studio e aguardar o sync do Gradle. O build padrão é o variant *debug*, que sobe o `MockWebServer` automaticamente em segundo plano.

Credenciais de demonstração. A tela de login oferece três botões que pulam o formulário e entram diretamente como cada perfil:

Botão	Perfil	E-mail simulado
Operador	OPERATOR	carlos.silva@empresa.com
Gestor	MANAGER	maria.santos@empresa.com
Liderança	LEADERSHIP	roberto.lima@empresa.com

Se preferir login via formulário, qualquer e-mail é aceito. O perfil é inferido pelo conteúdo do e-mail: contém "gestor" ou "maria" → gestor; contém "diretor", "lideranca" ou "roberto" → liderança; qualquer outro → operador.

Testes unitários. A camada de domínio possui testes unitários cobrindo `ScoreIdeaUseCase`, `CreateIdeaUseCase`, `ApproveIdeaUseCase`, `RejectIdeaUseCase`, `VoteIdeaUseCase` e `UpdateProjectUseCase`. Os testes rodam em JVM, sem necessidade de emulador.

Limpando o banco local. O `MockDataSeeder` é idempotente, controlado por uma flag em `SharedPreferences`. Para forçar um novo seed, basta limpar os dados do app pelas configurações do Android ou reinstalá-lo.

ADR-001: Adoção do Firebase como BaaS

Status: Aceito · **Data:** 22 de maio de 2026 · **Autor:** Time InnovaNexus

Contexto. A Sprint 1 do projeto exige integração com um BaaS escolhido entre Firebase, OneSignal, Backendless ou AWS Amplify, cobrindo autenticação real, persistência de dados, notificações push, automações de fluxo e dashboard agregado.

Decisão. Foi adotado o **Firebase**, combinando Authentication, Cloud Firestore, Cloud Messaging e Cloud Functions.

Alternativas consideradas:

Alternativa	Motivo da não escolha
OneSignal	Cobre apenas push. Exigiria outro BaaS para autenticação e persistência.
Backendless	SDK Android com pouca tração em Kotlin moderno; documentação escassa para a linguagem.
AWS Amplify	Cognito + AppSync + SNS + Lambda exigem configuração separada de cada serviço; curva de aprendizagem incompatível com o prazo da Sprint.

Consequências positivas:

- Um único SDK Kotlin-first cobre todos os requisitos.
- O free tier (plano Spark) é suficiente para um projeto acadêmico ou MVP.
- Console único, com instrumentação e logs unificados.
- O padrão offline-first do Firestore casa naturalmente com o cálculo local de dashboard já existente.

Consequências negativas e mitigações:

- **Lock-in** no Google Cloud — mitigado mantendo `RemoteDataSource` como interface abstrata. A eventual troca futura fica contida na camada de dados.
- **Consistência eventual** entre Room e Firestore — Firestore é definido como autoritativo para status e score das ideias; Room funciona como cache.
- **Custo das Cloud Functions** acima do free tier — a decisão será revisitada se o projeto atingir mais de 20 funções com lógica relevante.

Critério de revisão. A decisão deve ser reavaliada se: o projeto ultrapassar 20 Cloud Functions com lógica crítica, o volume de leituras no Firestore exceder 1 milhão por dia, ou houver necessidade de exportar dados para um data warehouse próprio.

Histórico da Sprint 1

A Sprint 1 foi entregue em 16 etapas, organizadas para construir a base técnica antes de evoluir para integrações externas.

Etapas de fundação técnica:

Etapas	Entrega
1	Mock-first com MockWebServer; fixtures JSON espelham os DTOs da API.
2	Dependências Firebase preparadas e ativadas condicionalmente pela presença de <code>google-services.json</code> .

3	Interfaces <code>RemoteDataSource</code> extraídas; repositórios refatorados para o padrão offline-first; modelo <code>Idea</code> ganha campos de score e votos.
4	UseCases de domínio (<code>ScoreIdeaUseCase</code> , <code>CreateIdeaUseCase</code> , <code>ApproveIdeaUseCase</code> , <code>RejectIdeaUseCase</code> , <code>VoteIdeaUseCase</code> , <code>UpdateProjectUseCase</code>).

Etapas de UI e integração com regras de negócio:

Etapas	Entrega
5	Novos ViewModels e ligação completa de UI com os UseCases.
6	Ranking dinâmico consumindo <code>UserRepository.observeRanking()</code> ; pódio com ocultação automática de slots vazios.
7	Dashboard com dados reais; <code>ideiasPerMonth</code> agregado a partir do Room; gráfico de linha com 12 pontos.
8	Testes unitários cobrindo os UseCases de domínio.
9	Documentação técnica em <code>docs/</code> com 8 documentos, ADR e Changelog.

Etapas de integração com Firebase:

Etapas	Entrega
11-13	Firebase real ativo; DataSources Firestore implementados; FCM recebendo notificações; switch lógico no <code>Application</code> decide entre Retrofit/Mock e Firebase em runtime.
15	Cloud Functions de automação (<code>onIdeaStatusChange</code> , <code>onIdeaVoteUpdated</code> , <code>onNewUser</code> , <code>aggregateDashboard</code>) prontas para deploy quando o plano Blaze for ativado.
16	Real-time sem Cloud Functions: <code>IdeaStatusObserver</code> via snapshot listener e <code>LocalNotifier</code> substituem o push server-side enquanto o projeto está no plano Spark.

Conclusão da Sprint. O InnovaNexus entrega, ao final da Sprint 1, uma plataforma funcional ponta a ponta em três modos (local, mock e Firebase), com regras de negócio testadas, dashboards funcionais, gamificação ativa e arquitetura preparada para evoluir sem reescrita.

PARTE V — APÊNDICES

Conclusão

O InnovaNexus entrega uma solução moderna para gestão da inovação corporativa, conectando ideias, projetos e resultados através de uma experiência mobile nativa para Android.

A combinação de **Clean Architecture**, **MVVM**, **Material Design 3** e integração com **Firebase** permitiu ao time entregar uma plataforma com três modos de operação, três perfis de usuário, regras de negócio testadas e indicadores executivos — tudo isso mantendo a base de código preparada para crescer e evoluir.

O projeto demonstra que decisões técnicas conscientes — separação de camadas, interfaces como pontos de extensão, testabilidade de regras de negócio em JVM, *offline-first* como padrão — geram valor mensurável: trocar o back-end de Retrofit/Mock para Firestore não exigiu uma única linha de mudança em UseCases ou Fragments.

A plataforma está pronta para a Sprint 2, que poderá focar em refinar a experiência do usuário, ativar Cloud Functions reais e expandir os indicadores de desempenho.